PET PROJECT · FASE SERVIDOR · ATIVIDADE AVALIATIVA 2

Sua primeira API com Node.js e Express

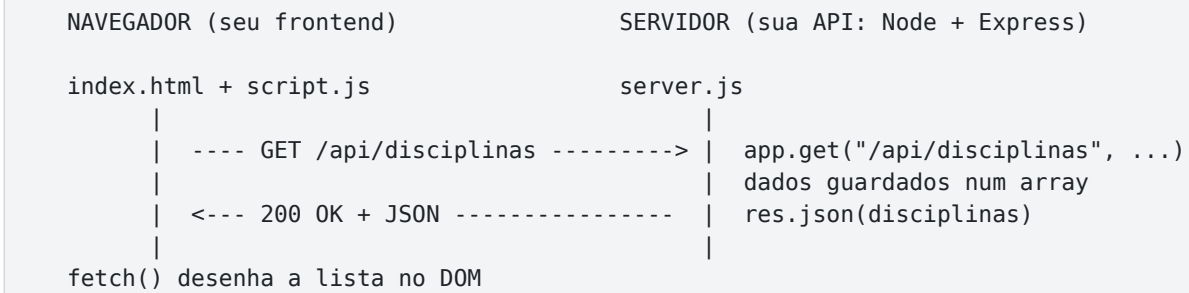
Tutorial passo a passo. Siga na ordem: cada passo só funciona se o anterior estiver funcionando.

O que você vai construir

No Laboratório 01 você construiu, com IA, a página **Meu Perfil Acadêmico**. Ela roda só no navegador, e os dados dela (seu nome, seu curso, suas disciplinas) estão escritos à mão dentro do HTML ou do JavaScript. Nesta fase do PET Project você vai criar o **servidor**: uma API que guarda os dados e os entrega ao seu frontend por HTTP, em formato JSON.

Hoje é o começo. Ao final desta aula, você deve ter uma API funcionando e o seu perfil sendo servido por ela, mostrando pelo menos uma lista que veio da API. Esta é a sua **Atividade Avaliativa 2**. Nas próximas aulas, o mesmo projeto cresce até a página inteira estar conectada.

O CAMINHO DE UMA REQUISIÇÃO



Os conceitos que você precisa

API é a porta de entrada de um programa para outro programa. Uma API web recebe requisições HTTP e responde com dados, quase sempre em JSON, em vez de responder com uma página pronta.

REST é um jeito de organizar essas portas. A ideia central: a URL diz *qual coisa* (o recurso) e o método HTTP diz *o que fazer* com ela. Por isso a mesma URL `/api/disciplinas` serve para listar (GET) e para cadastrar (POST).

Método	Exemplo	Significado
GET	`/api/disciplinas`	Listar todas
GET	`/api/disciplinas/2`	Buscar uma, pelo id
POST	`/api/disciplinas`	Criar uma nova (os dados vão no corpo da requisição)
PUT	`/api/disciplinas/2`	Alterar uma existente *(próximas aulas)*
DELETE	`/api/disciplinas/2`	Remover *(próximas aulas)*

Toda resposta HTTP vem com um **código de status**, que diz se deu certo antes mesmo de você ler o corpo:

Código	Quando usar
200 OK	Deu certo e estou devolvendo o que você pediu

Código	Quando usar
201 Created	Deu certo e criei um registro novo (resposta a um POST)
400 Bad Request	Você mandou dados inválidos ou faltando (erro do cliente)
404 Not Found	Isso que você pediu não existe
500 Internal Server Error	O servidor quebrou (erro no seu código)

PASSO 0

Confira se o Node está instalado

Abra o terminal do VS Code (menu **Terminal → Novo Terminal**) ou o PowerShell e digite:

```
node -v
npm -v
```

Se aparecerem dois números de versão, está tudo certo. Se aparecer *"node não é reconhecido como um comando"*, chame o professor antes de continuar.

PASSO 1

Crie o projeto e instale o Express

```
mkdir pet-api
cd pet-api
npm init -y
npm install express
```

`npm init -y` cria o arquivo `package.json`, a ficha do projeto. `npm install express` baixa o Express para a pasta `node_modules` e anota a dependência no `package.json`. Abra a pasta `pet-api` no VS Code (**Arquivo → Abrir Pasta**).

> **Atenção**
> Todos os comandos a partir daqui devem ser rodados **dentro da pasta** `pet-api`. O erro mais comum da aula é rodar `node server.js` na pasta errada.

PASSO 2

Seu primeiro servidor

Crie o arquivo `server.js` na pasta do projeto:

server.js

```
const express = require("express");

const app = express();
const PORTA = 3000;

app.get("/", (req, res) => {
  res.send("Minha API está no ar!");
});

app.listen(PORTA, () => {
  console.log(`Servidor rodando em http://localhost:${PORTA}`);
});
```

Rode no terminal e depois abra o endereço no navegador:

```
node --watch server.js
```

Leia o código assim: `app.get("/", ...)` quer dizer “quando chegar um GET no caminho /, rode esta função”. A função recebe `req` (a requisição que chegou) e `res` (a resposta que você vai devolver).

O `--watch` reinicia o servidor sozinho toda vez que você salvar o arquivo. Para parar o servidor, use **Ctrl+C** no terminal. Deixe esse terminal aberto rodando o servidor e, se precisar de outro comando, abra um segundo terminal.

PASSO 3

Os dados e a primeira rota da API

Por enquanto, os dados ficam num array na memória do servidor. Isso quer dizer que, se o servidor reiniciar, o que foi cadastrado some. Guardar em arquivo vem nas próximas aulas.

Apague a rota `"/"` do passo anterior e coloque isto no lugar, antes do `app.listen`:

server.js

```
let disciplinas = [
  { id: 1, nome: "Tecnologias de Internet", semestre: 4 },
  { id: 2, nome: "Banco de Dados", semestre: 3 }
];
let proximoId = 3;

app.get("/api/disciplinas", (req, res) => {
  res.json(disciplinas);
});

const perfil = { nome: "Seu Nome", curso: "Ciência da Computação", semestre: 4 };

app.get("/api/perfil", (req, res) => {
  res.json(perfil);
});
```

Abra **http://localhost:3000/api/disciplinas** no navegador. Você deve ver o array em JSON. Depois abra **/api/perfil**: ali vem um objeto só, não uma lista. Repare que você não precisou converter nada: o `res.json()` transforma o objeto JavaScript em texto JSON e avisa o navegador que aquilo é JSON.

O perfil é seu

Troque os dados de exemplo pelos do **seu** perfil: o seu nome e curso em `perfil`, e as disciplinas que você cursa de verdade. Se a sua página mostra outra lista (projetos, habilidades, certificados), pode usar ela no lugar das disciplinas. Cada item da lista precisa de um **id** único.

PASSO 4

Buscar um item pelo id

server.js

```
app.get("/api/disciplinas/:id", (req, res) => {
  const id = Number(req.params.id);
  const disciplina = disciplinas.find((d) => d.id === id);

  if (!disciplina) {
    return res.status(404).json({ erro: "Disciplina não encontrada" });
  }
  res.json(disciplina);
});
```

O `:id` na rota é um parâmetro: tudo que vier naquela posição da URL chega em `req.params.id`. Ele sempre chega como **texto**, por isso o `Number()`: sem ele, `"2" === 2` dá falso e a busca nunca encontra nada.

Teste **/api/disciplinas/1** (encontra) e **/api/disciplinas/99** (404). Abra o DevTools (F12), aba **Network**, recarregue e clique na requisição para ver o código de status.

PASSO 5

Cadastrar um item novo (POST)

Primeiro, logo depois de `const app = express();`, ensine o servidor a ler JSON no corpo das requisições:

server.js

```
app.use(express.json());
```

Depois, a rota:

server.js

```
app.post("/api/disciplinas", (req, res) => {
  const { nome, semestre } = req.body || {};

  if (!nome) {
    return res.status(400).json({ erro: "O campo nome é obrigatório" });
  }

  const nova = { id: proximoId++, nome, semestre };
  disciplinas.push(nova);
  res.status(201).json(nova);
});
```

O navegador só faz GET quando você digita uma URL na barra de endereço. Para testar o POST, abra o DevTools (F12) com a página **http://localhost:3000/api/disciplinas** aberta, vá na aba **Console** e cole:

```
fetch("/api/disciplinas", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ nome: "Redes de Computadores", semestre: 5 })
}).then((r) => { console.log(r.status); return r.json(); })
  .then((dados) => console.log(dados));
```

Deve aparecer **201** e o objeto criado, já com id 3. Recarregue a lista e confira. Depois teste mandando `{ semestre: 5 }` sem o nome: tem que voltar **400**.

> **Dica**
> Se o Chrome não deixar colar no console, digite `allow pasting` e aperte Enter antes.

PASSO 6

Coloque o seu perfil dentro do servidor

Crie uma pasta `public` dentro de `pet-api` e copie para ela o `index.html` do seu **Meu Perfil Acadêmico** (e o CSS, o JS e as imagens, se estiverem em arquivos separados). Se você fez o Laboratório 01 no CodePen ou no JSFiddle, copie de lá o HTML, o CSS e o JS para arquivos. Depois, logo abaixo do `express.json()`, adicione:

server.js

```
app.use(express.static("public"));
```

Abra **http://localhost:3000**. Seu perfil aparece, mas agora entregue pelo servidor, e não mais aberto direto do disco.

> **Repare na barra de endereço**
> Lembra da primeira aula? Antes você abria o arquivo e a barra mostrava `file:///C:/...`. Agora mostra `http://localhost:3000`: existe um servidor respondendo. É isso que permite ao seu JavaScript pedir dados para `/api/...`, porque a página e a API estão no mesmo endereço.

A estrutura da pasta deve ficar assim (o `node_modules` fica de fora do desenho):

```
pet-api/
├── package.json
├── server.js
└── public/
    ├── index.html
    ├── style.css      (se o seu CSS estiver separado)
    └── script.js      (se o seu JS estiver separado)
```

PASSO 7

A primeira conexão: a lista vem da API

Encontre no seu perfil a lista de disciplinas (ou a lista que você escolheu no passo 3), que hoje está escrita à mão no HTML. Apague os itens fixos e deixe só o elemento que os envolve, com um id. No exemplo: `<ul id="lista"></ul>`. No seu JavaScript, adicione o código abaixo. Se o seu JS está dentro do HTML, numa tag `<script>`, coloque lá mesmo:

public/script.js

```
async function carregarDisciplinas() {
  const resposta = await fetch("/api/disciplinas");
  const disciplinas = await resposta.json();

  const lista = document.getElementById("lista");
  lista.innerHTML = "";

  disciplinas.forEach((d) => {
    const li = document.createElement("li");
    li.textContent = `${d.nome} (${d.semestre}º semestre)`;
    lista.appendChild(li);
  });
}

carregarDisciplinas();
```

O `fetch` faz a mesma requisição que você fez pela barra de endereço no passo 3, só que pelo código. O `await` manda o código **esperar** a resposta chegar antes de seguir, porque ela vem pela rede e demora. É por isso que a função tem `async` na frente. Isso vai ser aprofundado em outra aula; por hoje, basta saber que sem o `await` você estaria tentando ler uma resposta que ainda não chegou.

Teste de fogo: cadastre um item novo pelo console (passo 5) e recarregue a página. O item novo tem que aparecer na lista, sem você ter mexido no HTML.

> **Terminou antes? Vá além**
> Faça o mesmo com o seu nome e curso: busque `/api/perfil` e coloque `perfil.nome` no `textContent` do título da página. Agora o perfil inteiro começa a vir do servidor.

Quando der errado

O que aparece	O que quase sempre é
`Cannot find module 'express'`	Você está na pasta errada, ou esqueceu o `npm install express`.
`EADDRINUSE :::3000`	Já tem um servidor rodando na porta 3000, provavelmente em outro terminal. Feche-o com Ctrl+C.
`Cannot GET /api/disciplina`	A rota não existe. Confira a URL letra por letra (singular × plural, barra a mais).
`req.body` vem vazio no POST	Faltou o `app.use(express.json())`, ou ele está **depois** das rotas, ou faltou o header `Content-Type`.
Salvei e nada mudou	O servidor foi iniciado sem `--watch`. Pare com Ctrl+C e rode de novo.
Página abre, mas a lista fica vazia	Abra o Console do DevTools. Confira se a página está em `localhost:3000` e não em `file://`, e se o id do elemento bate com o do JavaScript.

Entrega da Atividade Avaliativa 2

Entregue **até o fim da aula**, conforme a orientação do professor:

#	Item
1	A pasta `pet-api` compactada em .zip, **sem** a pasta `node_modules` (ela é recriada com `npm install`).
2	Print do navegador em **/api/disciplinas** (ou na sua lista) mostrando o JSON com os dados do **seu** perfil.
3	Print do Console mostrando o POST com status **201** e o objeto criado.
4	Print do seu perfil em **http://localhost:3000** mostrando a lista que veio da API, incluindo o item que você cadastrou no POST.

O que vem nas próximas aulas

PUT e DELETE, salvar os dados em arquivo para não sumirem quando o servidor reinicia, e conectar o resto do seu perfil à API. Guarde este projeto: é ele que vai crescer até a entrega final da fase servidor.